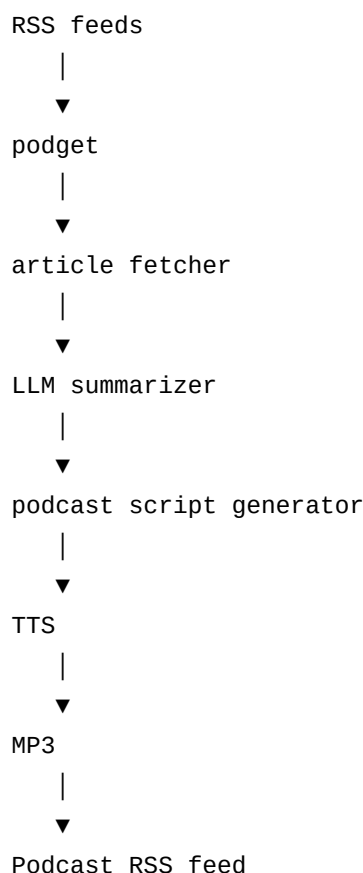


I would like to try `podget` with German news. My goal is to create a 5min podcast, ideally from sources like `swr3.de`. I'm a technical person who can script or code it. Can you recommend an approach?

Yes. If your goal is a **daily 5-minute German news podcast generated automatically from RSS feeds**, `podget` is a reasonable ingestion layer, but I would treat it as only the first step in a larger pipeline.

A robust architecture looks like this:



1. Collect stories with podget

SWR3 exposes an RSS feed:

- SWR3 RSS: <https://www.swr3.de/~rss/index.xml>  RSS Verzeichnis

You can create a `podget` channel list containing:

Bash

```
https://www.swr3.de/~rss/index.xml
```

and run:

Bash

```
podget
```

However, RSS descriptions are often too short for good summarization. I would use `podget` only to discover URLs, then fetch the article itself.

For example:

Python

```
feedparser  
requests  
trafilatura
```

Workflow:

Python

```
for item in rss_entries:  
    article = trafilatura.fetch_url(item.link)  
    text = trafilatura.extract(article)
```

2. Select only 3–5 stories

A 5-minute podcast is roughly:

- 650–800 spoken German words
- 3–5 stories

Prompt an LLM:

Pick the 4 most important stories.

Avoid duplicates.

Prioritize:

1. Germany
2. Europe
3. Major world events

3. Generate a podcast script

Don't ask the LLM for summaries only.

Ask for a radio-style script:

You are a public radio news editor.

Create a 5-minute German news briefing.

Structure:

- Intro (20 sec)
- Story 1
- Story 2
- Story 3
- Story 4
- Outro

Write for listening, not reading.

Use short sentences.

Avoid quoting articles.

This dramatically improves TTS quality.

Research on automatically generated news podcasts found that conversational or narrative formats are generally preferred over a simple sequence of article summaries.  arXiv

4. Generate speech

Good options:

Engine	Quality	Cost
OpenAI TTS	Excellent	Low
ElevenLabs	Excellent	Medium
Piper	Good	Free/self-hosted
Kokoro	Very good	Free/self-hosted

For a technical setup, I'd start with:

Python

```
openai.audio.speech.create(...)
```

or

Bash

```
piper
```

if you want full local generation.

5. Publish as a real podcast

After producing:

```
2026-06-11.mp3
```

generate a podcast RSS feed.

Podcast apps subscribe via RSS feeds containing episode metadata and audio enclosures.  podcaster.de +1

A simple approach:

Python

```
podgen
```

or generate XML yourself.

Host on:

- GitHub Pages
- S3
- Cloudflare R2
- your VPS

Then submit to podcast directories if desired.

6. My recommended stack

If I were building this today:

```
Cron
|
podget
|
Python
|
feedparser
trafilatura
|
GPT-5.5
|
OpenAI TTS
|
podgen
|
Cloudflare R2
```

Around 200–300 lines of Python is enough.

Alternative: Use existing SWR audio instead

Before building a news synthesis pipeline, check whether the existing SWR podcasts already fit your needs.

For example:

- SWR3 Tophthema provides a roughly four-minute daily news focus.

 My Podcast Data +1

- SWR3 Report provides longer background reporting.  SWR3

If your objective is learning and automation, building the pipeline is worthwhile. If your objective is simply "I want a 5-minute German briefing every morning", a custom RSS→LLM→TTS pipeline gives you much more control over topic selection and style than consuming existing news podcasts.

A nice enhancement is to pull from multiple German sources (SWR3, Tagesschau, Deutschlandfunk, ZEIT, Heise, etc.), rank stories, and generate a single "German Morning Briefing" episode every day. That tends to sound much more like a professional news roundup than a feed generated from one source alone.

    Sources

Could you implement 1 for <https://www.swr3.de/~rss/index.xml>?

Absolutely. For step 1 (RSS ingestion), I'd keep it very small and reliable.

Option A: Pure Python (simplest)

Install:

Bash

```
pip install feedparser
```

fetch_swr3.py :

Python

```
#!/usr/bin/env python3

import feedparser
from datetime import datetime

FEED_URL = "https://www.swr3.de/~rss/index.xml"

feed = feedparser.parse(FEED_URL)

print(f"Feed: {feed.feed.title}")
print()

for entry in feed.entries[:20]:
    published = entry.get("published", "")
    title = entry.get("title", "")
    link = entry.get("link", "")

    print("=" * 80)
    print(title)
    print(published)
    print(link)
```

Run:

Bash

```
python fetch_swr3.py
```

This gives you:

```
=====
Headline 1
Fri, ...
https://www.swr3.de/...
```

which is enough to feed the next stage.

RSS items typically contain a title, link, description, and publication date, making them ideal for news-ingestion workflows.  RSS... +1

Option B: Keep podget in the loop

If you specifically want to use podget:

Create ~/.podget/channels/swr3 :

```
https://www.swr3.de/~rss/index.xml
```

Run:

Bash

```
podget
```

Then parse the downloaded XML:

Python

```
import xml.etree.ElementTree as ET

rss_file = "/path/to/downloaded/index.xml"

root = ET.parse(rss_file).getroot()

for item in root.findall("./channel/item"):
    title = item.findtext("title")
    link = item.findtext("link")

    print(title)
    print(link)
    print()
```

Option C: Production-ready ingestion

This is the version I'd actually use for the podcast project:

Python


```
import feedparser
import json

FEED_URL = "https://www.swr3.de/~rss/index.xml"

feed = feedparser.parse(FEED_URL)

stories = []

for entry in feed.entries:
    stories.append({
        "title": entry.get("title"),
        "url": entry.get("link"),
        "published": entry.get("published"),
        "summary": entry.get("summary", ""),
    })

print(json.dumps(stories, ensure_ascii=False, indent=2))
```

Output:

JSON

```
[
  {
    "title": "...",
    "url": "https://...",
    "published": "...",
    "summary": "..."
  }
]
```

This JSON becomes the input to:

RSS

- fetch_swr3.py
- articles.json
- article downloader
- LLM summarizer
- podcast script
- TTS

For the next step, I'd probably skip `podget` entirely and build a single Python pipeline using:

Bash

```
pip install feedparser trafilatura openai
```

because once you start downloading article content, feedparser + trafilatura tends to be simpler than managing podget files.

    Sources